Module 4 – Introduction to DBMS

Introduction to SQL

Theory Questions:

1. **What is SQL, and why is it essential in database management?**

SQL (Structured Query Language) is a standard programming language used to interact with databases. It allows you to **create, read, update, and delete data** stored in a database.

- **Why SQL is essential in Database Management**

1. **Data Retrieval**
   - SQL helps you quickly fetch data using queries like `SELECT`.
2. **Data Manipulation**
   - You can insert, update, or delete data using commands like `INSERT`, `UPDATE`, and `DELETE`.
3. **Database Structure Management**
   - SQL is used to create and modify database structures (tables, schemas) using `CREATE`, `ALTER`, and `DROP`.
4. **Data Security**
   - It allows setting permissions and controlling access to data using commands like `GRANT` and `REVOKE`.
5. **Efficient Data Handling**
   - SQL can handle large volumes of data efficiently and perform complex operations like filtering, sorting, and joining tables.
6. **Standard Language**
   - SQL is widely used across different database systems like MySQL, Oracle, SQL Server, etc., making it a universal language for databases.

2. **Explain the difference between DBMS and RDBMS**.

Feature	DBMS	RDBMS
Full Form	Database Management System	Relational Database Management System
Data Storage	Stores data in files	Stores data in tables
Relationship	No relationship between data	Data is related using keys
Structure	Simple structure	More organized structure
Data Redundancy	Higher	Lower
Examples	File system, XML	MySQL, Oracle, SQL Server

3. Describe the role of SQL in managing relational databases.

Role of SQL in Managing Relational Databases (Simple):

- **Create tables:** SQL is used to create database tables (`CREATE`)
- **Store data:** Insert data into tables (`INSERT`)
- **Retrieve data:** Get data using queries (`SELECT`)
- **Update data:** Modify existing data (`UPDATE`)
- **Delete data:** Remove data (`DELETE`)
- **Manage relationships:** Use keys and joins to connect tables

4. What are the key features of SQL?

Key Features of SQL (Simple):

- **Easy to use:** Simple and readable language
- **Data Querying:** Retrieve data using `SELECT`
- **Data Manipulation:** Insert, update, delete data (`INSERT`, `UPDATE`, `DELETE`)
- **Data Definition:** Create and modify tables (`CREATE`, `ALTER`, `DROP`)
- **Security:** Control access using `GRANT` and `REVOKE`
- **Data Control:** Manage transactions using `COMMIT` and `ROLLBACK`

2. SQL Syntax

Theory Questions:

1. What are the basic components of SQL syntax?

Basic Components of SQL Syntax (Simple):

- **Keywords:** Special words like `SELECT`, `INSERT`, `UPDATE`, `DELETE`
- **Table Name:** The name of the table you are working with
- **Columns:** The fields (data) you want to access
- **Conditions:** Use `WHERE` to filter data
- **Operators:** Symbols like `=`, `>`, `<`, `AND`, `OR`

2. Write the general structure of an SQL SELECT statement.

```
SELECT column1, column2

FROM table_name

WHERE condition

GROUP BY column_name

HAVING condition
```

ORDER BY column_name;

3. Explain the role of clauses in SQL statements.

Role of Clauses in SQL Statements (Simple):

Clauses are parts of an SQL query that **define what data to select and how to display it**.

👉 **Main roles:**

- **SELECT:** chooses the columns
- **FROM:** specifies the table
- **WHERE:** filters the data (condition)
- **GROUP BY:** groups data
- **HAVING:** filters grouped data
- **ORDER BY:** sorts the result

3. SQL Constraints

Theory Questions:

1. What are constraints in SQL? List and explain the different types of constraints.

Constraints in SQL (Simple):
Constraints are **rules applied to table columns** to ensure the data is **accurate and valid**.

🔹 Types of Constraints:

- **NOT NULL**
  → Ensures a column **cannot have empty (NULL) values**
- **UNIQUE**
  → Ensures all values in a column are **different**
- **PRIMARY KEY**
  → Uniquely identifies each record (combines **NOT NULL + UNIQUE**)
- **FOREIGN KEY**
  → Links one table to another (maintains **relationship**)
- **CHECK**
  → Ensures values meet a **specific condition** (e.g., age > 18)
- **DEFAULT**
  → Sets a **default value** if no value is given

2. How do PRIMARY KEY and FOREIGN KEY constraints differ?

Feature	PRIMARY KEY	FOREIGN KEY
Purpose	Uniquely identifies each record	Links one table to another
Uniqueness	Must be unique	Can have duplicate values
NULL Values	Not allowed	Can allow NULL
Table	Exists in the same table	Refers to another table
Number	Only one primary key per table	Can have multiple foreign keys

3. What is the role of NOT NULL and UNIQUE constraints?

Role of NOT NULL and UNIQUE Constraints (Simple):

- **NOT NULL**
  → Ensures that a column **cannot have empty (NULL) values**
  → Every record must have a value in that column
- **UNIQUE**
  → Ensures that all values in a column are **different (no duplicates)**

4. Main SQL Commands and Sub-commands (DDL)
   Theory Questions:

   1. Define the SQL Data Definition Language (DDL).

   SQL Data Definition Language (DDL) (Simple):

DDL is a part of SQL used to **define and manage the structure of a database**.

👉 It is used to:

- Create tables
- Modify tables
- Delete tables

👉 Common DDL commands:

- `CREATE`
- `ALTER`
- `DROP`
- `TRUNCATE`

2. Explain the CREATE command and its syntax.

CREATE Command in SQL (Simple):

The `CREATE` command is used to **create a new database or table**.

🔹 Syntax (for creating a table):

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    column3 datatype
);
```

🔹 Example:

```
CREATE TABLE student (
    id INT,
    name VARCHAR(50),
    age INT
);
```

3. What is the purpose of specifying data types and constraints during table creation?

Purpose of Data Types and Constraints (Simple):

- **Data Types**
  → Define what kind of data a column can store (like `INT`, `VARCHAR`)
  → Help store data in the **correct format**
- **Constraints**
  → Set rules on data (like `NOT NULL`, `UNIQUE`)
  → Ensure data is **accurate and valid**

5. ALTER Command
   Theory Questions:

1. What is the use of the ALTER command in SQL?

ALTER Command in SQL (Simple):

The `ALTER` command is used to **modify an existing table structure**.

👉 It can be used to:

- **Add** new columns
- **Delete** columns
- **Change** data types
- **Rename** columns

2. How can you add, modify, and drop columns from a table using ALTER?

Using ALTER to Add, Modify, and Drop Columns (Simple):

🔹 Add a Column

```
ALTER TABLE table_name
ADD column_name datatype;
```

🔹 Modify a Column

```
ALTER TABLE table_name
MODIFY column_name new_datatype;
```

🔹 Drop a Column

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

6. DROP Command

Theory Questions:

1. What is the function of the DROP command in SQL?

DROP Command in SQL (Simple):

The `DROP` command is used to **delete a table or database completely**.

👉 It removes:

- Table structure
- All data inside the table

🔹 Example:

```
DROP TABLE table_name;
```

2. What are the implications of dropping a table from a database?

Implications of Dropping a Table (Simple):

- **All data is deleted**
  → Every record in the table is permanently removed
- **Table structure is removed**
  → Columns, constraints, and indexes are deleted
- **Cannot be recovered easily**
  → Data is lost unless you have a backup
- **Affects related tables**
  → If linked with foreign keys, it may cause errors or break relationships

7. Data Manipulation Language (DML)
   Theory Questions:

1. Define the INSERT, UPDATE, and DELETE commands in SQL

INSERT, UPDATE, and DELETE in SQL (Simple):

- **INSERT**
  → Used to **add new data** into a table
- **UPDATE**
  → Used to **change existing data** in a table
- **DELETE**
  → Used to **remove data** from a table

2. What is the importance of the WHERE clause in UPDATE and DELETE operations?

Importance of WHERE Clause in UPDATE and DELETE (Simple):

- The `WHERE` clause is used to **specify which records to update or delete**

👉 **Why it is important:**

- It prevents **updating or deleting all records**
- It targets **specific rows only**
- Helps avoid **data loss or mistakes**

🔹 **Example:**

```
UPDATE student
SET age = 20
WHERE id = 1;
```

8. Data Query Language (DQL)
   Theory Questions:

1. What is the SELECT statement, and how is it used to query data?

SELECT Statement in SQL (Simple):

The `SELECT` statement is used to **retrieve (get) data from a database**.

👉 **How it is used:**

- You choose the **columns** you want
- You specify the **table**
- You can use conditions with `WHERE`

🔹 Example:

```
SELECT name, age
FROM student
WHERE age > 18;
```

2. Explain the use of the ORDER BY and WHERE clauses in SQL queries.

Use of ORDER BY and WHERE in SQL (Simple):

- **WHERE clause**
  → Used to **filter data** based on a condition
  → Example: show only records where age > 18
- **ORDER BY clause**
  → Used to **sort data** in ascending (`ASC`) or descending (`DESC`) order

🔹 Example:

```
SELECT *
FROM student
WHERE age > 18
ORDER BY age DESC;
```

9. Data Control Language (DCL)
   Theory Questions:

1. What is the purpose of GRANT and REVOKE in SQL?

Purpose of GRANT and REVOKE in SQL (Simple):

- **GRANT**
  → Used to **give permissions** to users
  → Example: allow a user to SELECT or INSERT data
- **REVOKE**
  → Used to **remove permissions** from users

2. How do you manage privileges using these commands?

Managing Privileges using GRANT and REVOKE (Simple):

- **Using GRANT**
  → Give specific permissions to a user

```
GRANT SELECT, INSERT ON table_name TO user_name;
```

- **Using REVOKE**
  → Remove permissions from a user

```
REVOKE INSERT ON table_name FROM user_name;
```

👉 You can control permissions like:

- `SELECT` (read data)
- `INSERT` (add data)
- `UPDATE` (modify data)
- `DELETE` (remove data)

10.Transaction Control Language (TCL)

Theory Questions:

1. What is the purpose of the COMMIT and ROLLBACK commands in SQL?

Purpose of COMMIT and ROLLBACK in SQL (Simple):

- **COMMIT**
  → Saves all changes made in the database permanently
- **ROLLBACK**
  → Undoes changes and returns the database to the previous state

2. Explain how transactions are managed in SQL databases.

ransactions in SQL (Simple):

A **transaction** is a group of SQL operations treated as **one unit**.

👉 **How transactions are managed:**

- **BEGIN / START TRANSACTION** → starts the transaction
- **COMMIT** → saves all changes permanently
- **ROLLBACK** → cancels changes if something goes wrong

👉 **Why it is important:**

- Ensures data is **correct and consistent**
- Prevents **partial updates** (all or nothing)

11.SQL Joins

Theory Questions:

1. Explain the concept of JOIN in SQL. What is the difference between INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN?

JOIN in SQL (Simple):
A **JOIN** is used to **combine data from two or more tables** based on a related column.

🔹 Types of JOIN:

JOIN Type	Description
INNER JOIN	Returns only **matching records** from both tables
LEFT JOIN	Returns **all records from left table** and matching from right
RIGHT JOIN	Returns **all records from right table** and matching from left
FULL OUTER JOIN	Returns **all records from both tables** (matched + unmatched)

2. How are joins used to combine data from multiple tables?

How JOINs Combine Data from Multiple Tables (Simple):

JOINs are used to **link tables using a common column** (like `id`).

👉 **How it works:**

- You select data from **two tables**
- Use a **JOIN condition** (usually with `ON`)
- Match rows where values are equal

🔹 Example:

```
SELECT student.name, courses.course_name
FROM student
JOIN courses
ON student.cid = courses.cid;
```

👉 This will combine data where **student.cid = courses.cid**

12.SQL Group By

Theory Questions:

1. What is the GROUP BY clause in SQL? How is it used with aggregate functions?

GROUP BY in SQL (Simple):

The `GROUP BY` clause is used to **group rows that have the same values** in a column.

👉 It is commonly used with **aggregate functions** like:

- `COUNT()` → count records
- `SUM()` → total values
- `AVG()` → average
- `MAX()` / `MIN()` → highest / lowest

🔹 **Example:**

```
SELECT cid, COUNT(*)
FROM student
GROUP BY cid;
```

👉 This groups students by `cid` and counts them.

2. Explain the difference between GROUP BY and ORDER BY.

Difference between GROUP BY and ORDER BY (Simple):

Feature	GROUP BY	ORDER BY
Purpose	Groups rows with same values	Sorts the result
Use	Used with aggregate functions	Used to arrange data
Result	Reduces rows into groups	Keeps all rows, just changes order
Example	`GROUP BY cid`	`ORDER BY name ASC`

13.SQL Stored Procedure

Theory Questions:

1. What is a stored procedure in SQL, and how does it differ from a standard SQL query?

Stored Procedure in SQL (Simple):

A **stored procedure** is a **pre-written set of SQL statements** stored in the database that can be executed whenever needed.

🔹 **Difference from a Normal SQL Query:**

Feature	Stored Procedure	Standard SQL Query
Definition	Saved in database	Written and executed each time
Reusability	Can be reused many times	Not reusable (written again)
Performance	Faster (precompiled)	Slower (runs each time)
Complexity	Can handle complex logic	Usually simple queries

2. Explain the advantages of using stored procedures.

Advantages of Stored Procedures (Simple):

- **Reusable**
  → Write once and use many times

- **Faster Performance**
  → Precompiled, so execution is quicker
- **Security**
  → Users can access procedures without direct access to tables
- **Less Code**
  → Reduces repeated SQL code
- **Handles Complex Logic**
  → Can include conditions, loops, and multiple queries

14.SQL View

Theory Questions:

1. What is a view in SQL, and how is it different from a table?

View in SQL (Simple):

A **view** is a **virtual table** created from a SQL query. It shows data from one or more tables but **does not store data itself**.

🔹 Difference between View and Table:

Feature	View	Table
Data Storage	Does not store data	Stores actual data
Type	Virtual table	Real table
Creation	Based on a query	Created directly
Space	Does not take much space	Takes storage space
Usage	Used for security and simplicity	Used to store data

2. Explain the advantages of using views in SQL databases.

Advantages of Views in SQL (Simple):

- **Security**
  → Show only selected data to users
- **Simplicity**
  → Complex queries can be saved as a simple view
- **Data Abstraction**
  → Hide complex table structure
- **Reusability**
  → Use the same query multiple times
- **Consistency**
  → Always shows updated data from tables

15.SQL Triggers

Theory Questions:

1. What is a trigger in SQL? Describe its types and when they are used.

Trigger in SQL (Simple):

A **trigger** is a **special SQL procedure** that runs **automatically** when an event (like INSERT, UPDATE, DELETE) occurs on a table.

🔹 Types of Triggers:

- **BEFORE Trigger**
  → Runs **before** the operation
  → Used to **validate or check data**
- **AFTER Trigger**
  → Runs **after** the operation
  → Used to **log changes or update related data**
- **INSTEAD OF Trigger**
  → Runs **instead of the operation**
  → Used mostly with **views**

🔹 When they are used:

- To **automatically enforce rules**
- To **maintain data consistency**
- To **track or log changes**

2. Explain the difference between INSERT, UPDATE, and DELETE triggers.

Difference between INSERT, UPDATE, and DELETE Triggers (Simple):

Trigger Type	When it Executes	Purpose
INSERT Trigger	When new data is added	To check or act on new records
UPDATE Trigger	When existing data is modified	To track or validate changes
DELETE Trigger	When data is removed	To log or handle deleted records

16.Introduction to PL/SQL

Theory Questions:

1. What is PL/SQL, and how does it extend SQL's capabilities?

PL/SQL (Simple):

PL/SQL (Procedural Language/SQL) is an extension of SQL used in databases like Oracle.

👉 **How it extends SQL: 2. List and explain the benefits of using PL/SQL.**

- Adds **programming features** like loops and conditions (`IF`, `LOOP`)
- Allows writing **blocks of code** (procedures, functions)
- Helps handle **errors (exception handling)**
- Supports **complex operations** in one program

2. List and explain the benefits of using PL/SQL.

Benefits of PL/SQL (Simple):

- **Better Performance**
  → Executes multiple SQL statements in one block
- **Reusability**
  → Code can be reused using procedures and functions
- **Error Handling**
  → Handles errors using exception handling
- **Security**
  → Restricts direct access to data
- **Supports Programming Logic**
  → Allows use of loops, conditions (`IF`, `LOOP`)

17. PL/SQL Control Structures

Theory Questions:

1. What are control structures in PL/SQL? Explain the IF-THEN and LOOP control structures.

Control Structures in PL/SQL (Simple):
Control structures are used to **control the flow of execution** in a PL/SQL program.

🔹 IF-THEN

- Used to **make decisions**
- Executes code only if a condition is true

```
IF condition THEN
   -- statements
END IF;
```

🔹 LOOP

- Used to **repeat a block of code**
- Runs until a condition is met

```
LOOP
   -- statements
   EXIT WHEN condition;
END LOOP;
```

2. How do control structures in PL/SQL help in writing complex queries?

Role of Control Structures in PL/SQL (Simple):

Control structures help to **handle complex logic** in SQL programs.

👉 **How they help:**

- **IF-THEN** → Make decisions based on conditions
- **LOOP** → Repeat tasks multiple times
- **Better control** → Execute steps in a proper order
- **Handle complex tasks** → Combine multiple operations in one block

18.SQL Cursors

Theory Questions:

1. What is a cursor in PL/SQL? Explain the difference between implicit and explicit cursors.

Cursor in PL/SQL (Simple):
A **cursor** is a pointer used to **fetch and process data row by row** from a query result.

🔹 Types of Cursors:

Feature	Implicit Cursor	Explicit Cursor
Definition	Automatically created by SQL	Created manually by user
Usage	For simple queries	For complex queries
Control	Less control	More control
Example	`SELECT`, `INSERT` (auto handled)	Defined using `DECLARE`, `OPEN`, `FETCH`, `CLOSE`

2. When would you use an explicit cursor over an implicit one?

When to Use an Explicit Cursor (Simple):

Use an **explicit cursor** when you need **more control over data processing**.

👉 **You use it when:**

- You want to **process rows one by one**
- You need to **apply complex logic** on each row
- You want to **control opening, fetching, and closing** of data
- When a query returns **multiple rows** and needs detailed handling

19.Rollback and Commit Savepoint

Theory Questions:

1. Explain the concept of SAVEPOINT in transaction management. How do ROLLBACK and COMMIT interact with savepoints?

What is SAVEPOINT?

A **SAVEPOINT** is like a **checkpoint** inside a transaction.

👉 It lets you **go back to a certain point** instead of cancelling everything.

◇ Example (Easy)

```
BEGIN;

INSERT INTO student VALUES (1, 'Raj');
SAVEPOINT A;

INSERT INTO student VALUES (2, 'Amit');
SAVEPOINT B;

INSERT INTO student VALUES (3, 'Neha');
```

◇ ROLLBACK with SAVEPOINT

```
ROLLBACK TO A;
```

👉 This will:

- Remove **Amit and Neha**
- Keep **Raj**

✔ Only changes **after SAVEPOINT A are undone**

◇ COMMIT

```
COMMIT;
```

👉 This will:

- Save all data permanently
- Delete all savepoints

❌ After COMMIT, you **cannot rollback**

◇ Very Simple Idea

- **SAVEPOINT** → mark a point
- **ROLLBACK TO** → go back to that point
- **COMMIT** → save everything forever

🎯 Easy Example (Real Life)

Think of writing a document:

- SAVEPOINT = saving a draft
- ROLLBACK = go back to old draft
- COMMIT = final submit (no changes allowed)

2. When is it useful to use savepoints in a database transaction?

☑ 1. In Large Transactions

When a transaction has many steps:

- You can go back to a **specific step**
- No need to cancel the whole work

👉 Example: Adding multiple records in a database

☑ 2. When Errors May Occur

If some part of the transaction might fail:

- Use SAVEPOINT before that step
- Rollback only the failed part

☑ 3. Partial Undo Needed

When you want to:

- Keep some changes
- Remove only unwanted changes

☑ 4. Complex Operations

In operations like:

- Banking transactions
- Order processing systems

👉 SAVEPOINT helps manage each step safely